

Devoir de logique

Ahmat Mahamat Ahmat (21912949)

March 19, 2024

J'ai travaillé avec les graphes simples non orientés

1 Question 1

1.1 TRIANGLE

La propriété **TRIANGLE** est défini par :

$$\exists x \exists y \exists z [E(x, y) \wedge E(y, z) \wedge E(z, x)]$$

1.2 Classe préfixe décidable de TRIANGLE

TRIANGLE appartient à la classe de **Bernays-Schönfinkel**

1.3 La fonction triangle

```
1 def triangle(self):
2     for i in range(self.n):
3         for j in self.voisinage(i):
4             for k in self.voisinage(j):
5                 if i in self.voisinage(k):
6                     return True
7     return False
```

2 Question 2

2.1 La fonction construction triangle

```
1 def construction_triangle(n):
2     g = Graphe(n)
3     while not g.triangle():
4         g.ajoute_arete()
5     return g.m
```

3 Question 3

3.1 Histogramme de la fonction construction triangle

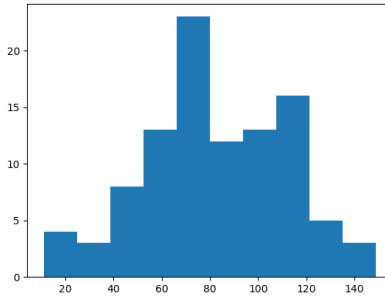


Figure 1: Histogramme de **TRIANGLE**

4 Question 4

4.1 CHEMIN2

La propriété CHEMIN2 est défini par :

$$\exists x \forall y [x \neq y \rightarrow \exists z (E(x, z) \wedge E(z, y) \wedge x \neq z \wedge y \neq z)]$$

4.2 Classe préfixe décidable de CHEMIN2

CHEMIN2 appartient à la classe d'[Ackermann](#)

4.3 La fonction chemin2

```
1 def chemin2(self):
2     for x in range(self.n):
3         tous_les_y = True
4         for y in range(self.n):
5             if x != y:
6                 chemin2_trouve = False
7                 for z in self.voisinage(x):
8                     if z != x and z != y and y in self.voisinage(z):
9                         chemin2_trouve = True
10                    break
11                if not chemin2_trouve:
12                    tous_les_y = False
13                    break
14            if tous_les_y:
15                return True
16    return False
```

5 Question 5

5.1 La fonction construction chemin

```
1 def construction_chemin(n):  
2     g = Graphe(n)  
3     while not g.chemin2():  
4         g.ajoute_arete()  
5     return g.m
```

6 Question 6

6.1 Histogramme de la fonction construction chemin

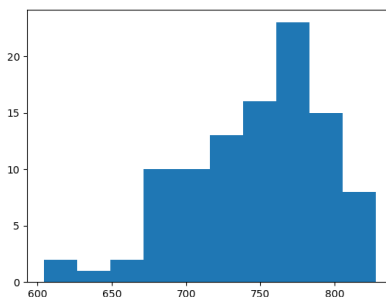


Figure 2: Histogramme de **CHEMIN2**

7 Question 7

7.1 Démonstration par récurrence

Nous souhaitons démontrer que la fonction $R_1(i)$, définie par $R_1(i) = R_2(i, 0)$, satisfait la relation suivante $\forall i \in \mathbb{N}$:

$$R_1(i) = \frac{(i+1)(i+2)}{2} - 1$$

Étape de base

Pour $i = 0$, nous avons:

$$R_1(0) = \frac{(0+1)(0+2)}{2} - 1 = 0$$

Ce qui correspond à la définition de $R_2(0, 0) = 0$. Ainsi, la propriété est vraie pour $i = 0$.

Hypothèse de récurrence

Supposons que la propriété est vraie pour un certain $i = k$, c'est-à-dire que :

$$R_1(k) = \frac{(k+1)(k+2)}{2} - 1$$

Étape d'induction

Nous devons montrer que la propriété est vraie pour $i = k + 1$:

$$\begin{aligned} R_1(k+1) &= R_2(k+1, 0) \\ &= R_2(k+0, 0) + (k+1) \\ &= R_1(k) + k+1 \\ &= \left[\frac{(k+1)(k+2)}{2} - 1 \right] + k+1 \\ R_1(k+1) &= \frac{(k+2)(k+3)}{2} - 1 \end{aligned}$$

Doù, d'après le principe récurrence, $\forall i \in \mathbb{N}$ alors $R_1(i) = \frac{(i+1)(i+2)}{2} - 1$

8 Question 8

8.1 Vérification des premiers termes de S1 et S2

```
1 def R1(i):
2     return ((i + 1) * (i + 2)) // 2 - 1
3
4 def S1(k):
5     return math.floor(math.sqrt(2 * k + 2)) - 1
6
7 def S2(k):
8     if k == 0: return (0, 0)
9     p = S1(k)
10    i = k - R1(p - 1) - 1
11    if p - i >= 0:
12        j = p - i
13    else:
14        i = k - R1(p) - 1
15        j = p + 1 - i
16    return (i, j)
```

Les premiers termes de $S_1(k)$ pour $k = 0$ jusqu'à $k = 9$ sont :

[0, 1, 1, 1, 2, 2, 2, 3, 3, 3]

Et les premiers termes de $S_2(k)$ pour $k = 0$ jusqu'à $k = 9$ sont :

[(0, 0), (0, 1), (1, 0), (0, 2), (1, 1), (2, 0), (0, 3), (1, 2), (2, 1), (3, 0)]

9 Question 9

```
1 def generation_terme(k):
2     if k == 1:
3         return 'x'
4     elif k == 2:
5         return 'y'
6     else:
7         i, j = S2(k - 3)
8         return f'f({generation_terme(i+1)},{generation_terme(j+1)})'
```